Program Studi Teknik Elektro ITB

Nama Kuliah (Kode) : EL2008-Pemecahan Masalah dengan Pemrograman
Tahun / Semester : 2025-2026 / Genap
Modul : 5 - Foundation of Algorithm
Nama Asisten / NIM : Naufal Afiq Muzaffar / 13222025
Nama Praktikan / NIM : Muhammad Aqeel Ghani / 13224071

Buku Catatan Laboratorium

- **Alasan pemilihan soal praktikum dan strategi awal / rancangan algoritma.**

  Alasan saya dalam memilih soal ini adalah saya cukup familiar dengan adjacency matrix dari matakuliah Struktur Diskrit. Lalu setelah saya membaca soal beberapa kali, saya menyadari bahwa saya tidak perlu membuat matrix, melainkan array of int yang menghitung degree tiap vertex sudah cukup untuk menjawab array.

- **Kejadian selama praktikum**

  Selama praktikum sempat terjadi kebingungan saat pada awal masih berpikir untuk menggunakan matriks, terutama bagaimana melakukan passing parameter pada matriks dengan size yang belum diketahui. Tetapi akhirnya setelah merubah strategi untuk menghitung degree langsung saat menerima input dan langsung dimasukan ke array satu dimensi.

- **Tahapan strategi (milestone) dan proses semua proses debugging yang dilakukan untuk menyelesaikan masalah.**

  Pada Awalnya, saya berpikir untuk menerima input dalam bentuk matriks dua dimensi. Lalu membuat fungsi untuk menghitung degree secara terpisah. Namun, setelah dipikirkan kembali, saya memilih untuk langsung menghitung degree saat menerima input dan langsung menyimpannya dalam suatu array.

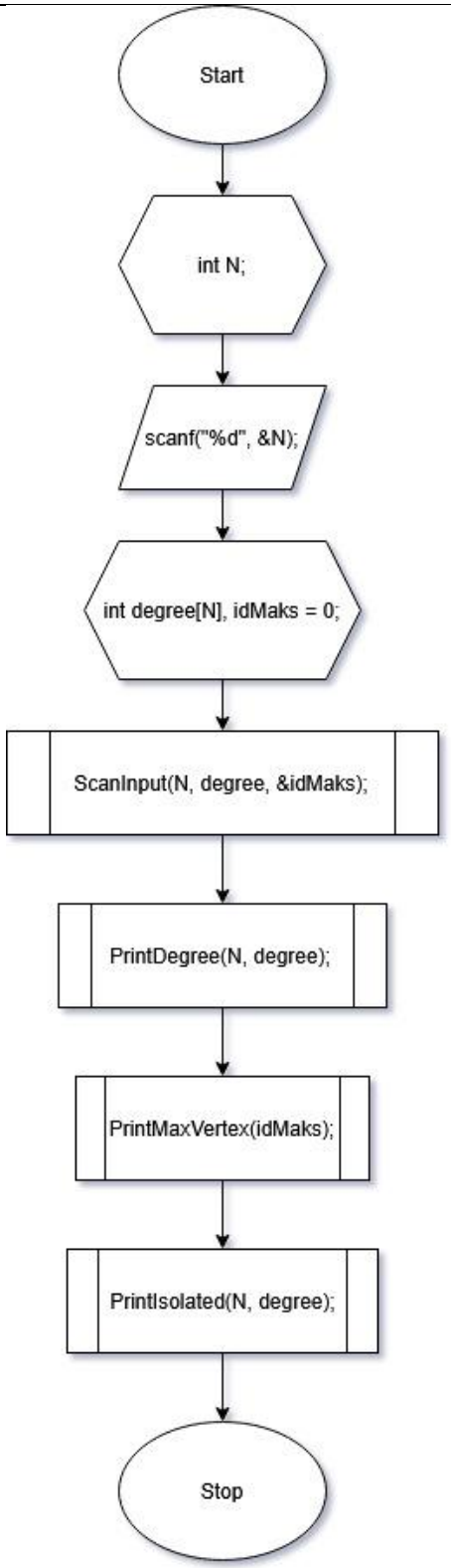
  Lalu saya juga mencari nilai vertex max dengan membandingkan nilai maks yang sudah didapat dengan nilai degree di vertex yang baru saja dihitung. Jika nilai degree di vertex yang baru lebih besar dibanding nilai degree max sebelumnya, maka indeks vertex baru ini di simpan menggantikan indeks vertex sebelumnya.

  Fungsi print degree akan mengambil degree yang sudah disimpan di array untuk ditampilkan beserta Indeksnya.

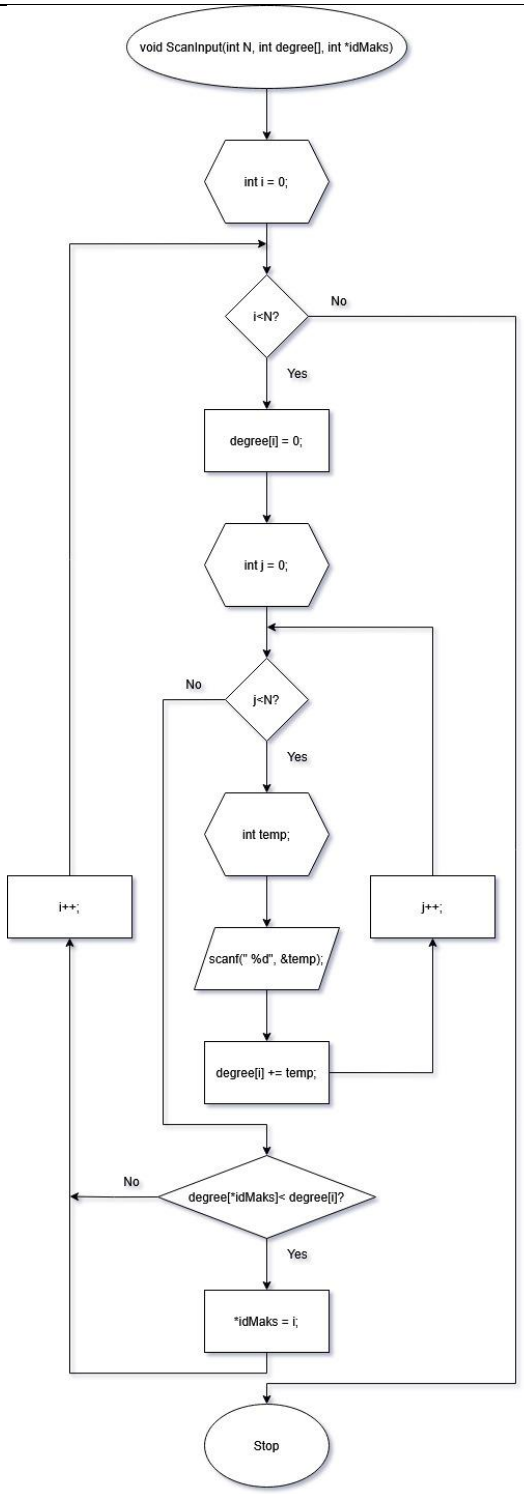
  Fungsi print max vertex akan menampilkan indeks verteks dengan degree terbesar (dan indeks terkecil).

Sedangkan fungsi print isolated akan menampilkan indeks yang memiliki degree nol. Jika tidak ada verteks dengan degree nol, maka akan ditampilkan None.

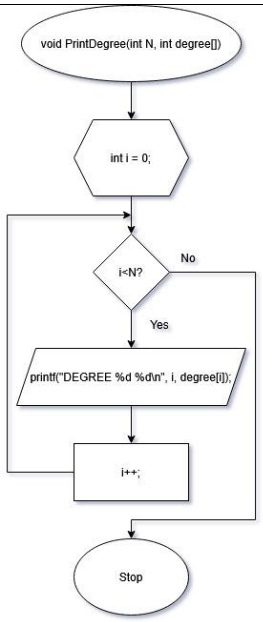
- **Flowchart dari kode yang dibuat.**



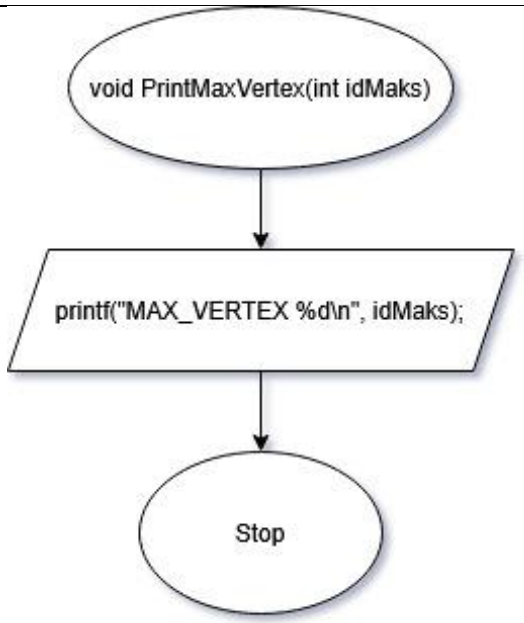
Gambar 1 Flowchart Main



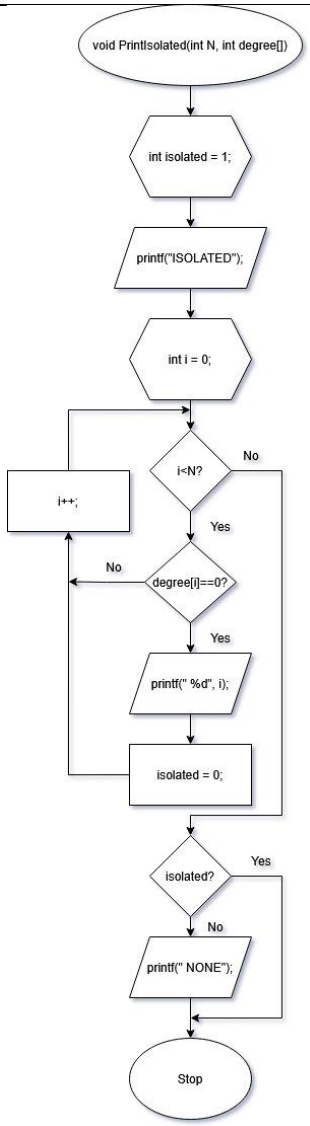
Gambar 2 Flowchart Scan Input



Gambar 3 Flowchart Print Degree

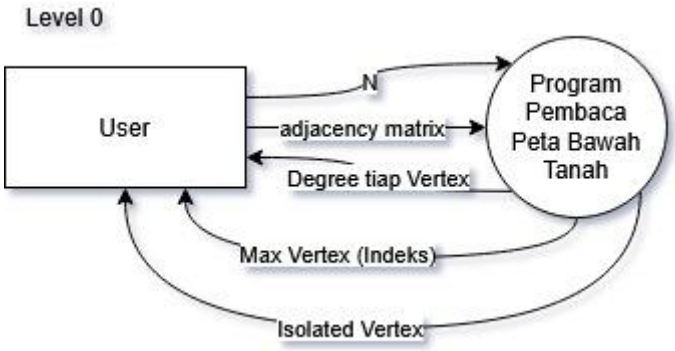


Gambar 4 Flowchart Print Max Vertex

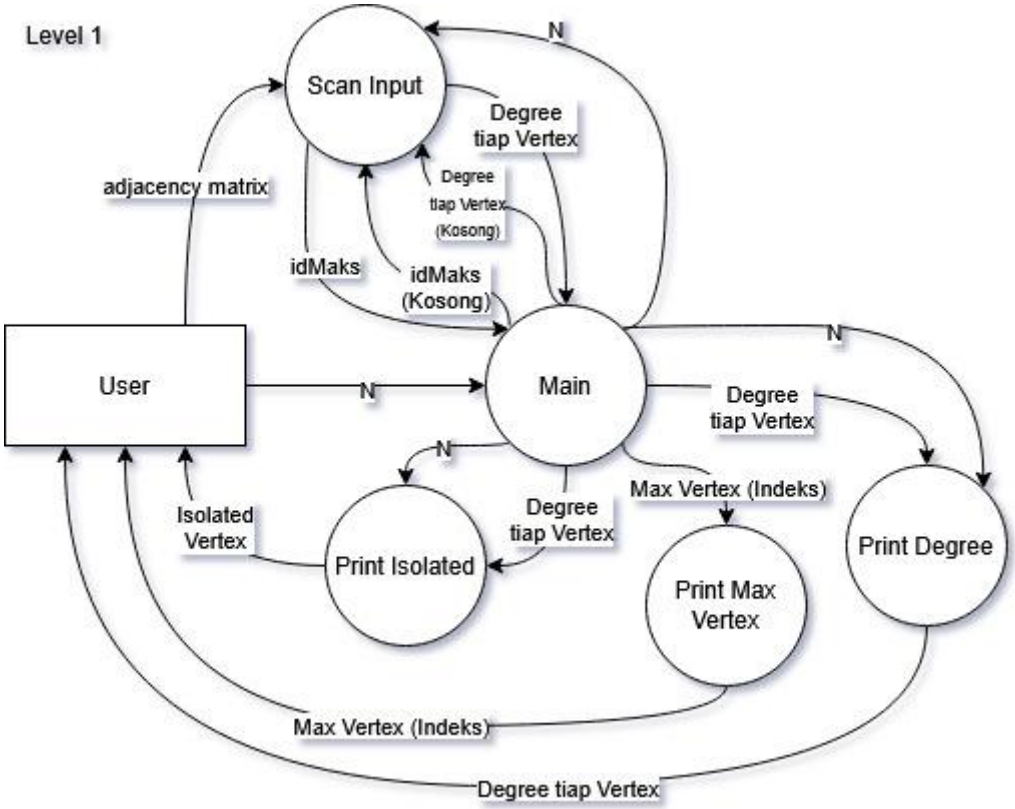


Gambar 5 Flowchart Print Isolated

- **DFD dari kode yang dibuat (minimal hingga DFD level 1)**



Gambar 6 DFD Level 0



Gambar 7 DFD Level 1

- **Analisis kompleksitas ruang dan waktu dari kode yang dibuat (hanya ada setelah materi sudah diajarkan di kelas)**

  Komplesitas waktu program ini adalah $O(n^2)$ dikarenakan terdapat loop dalam loop dalam fungsi Scan Input. Sedangkan program ini memiliki komplesitas ruang O(n) dikarenakan hanya menggunakan array satu dimensi.

Lampiran

Github BCL Modul Muhammad Aqeel Ghani :

github.com/AqeelGhani/BCL-PMP-Modul-5-13224071

```c
/** EL2008 Praktikum Pemecahan Masalah dengan Pemrograman 2025/2026
 *   Modul               : 5 - Foundation of Algorithm
 *   Hari dan Tanggal    : Senin, 11 Mei 2026
 *   Nama (NIM)          : Muhammad Aqeel Ghani (13224071)
 *   Nama File           : soal1.c
 *   Deskripsi           : Membaca peta fasilitas bawah tanah dengan menghitung derajat setiap simpul, simpul maksimum, dan simpul terisolasi
 *
 */
// Penambahan standard input output
#include <stdio.h>

// Prosedur Scan Input (Membaca Degree Tiap vertex dan mencari max vertex)
void ScanInput(int N, int degree[], int *idMaks){
    for(int i = 0; i<N; i++){
        // Inisialisasi degree tiap vertex
        degree[i] = 0;
        for(int j = 0; j<N; j++){
            int temp;
            // membaca adjency matrix
            scanf(" %d", &temp);
            // Menghitung degree vertex
            degree[i] += temp;

        }
        // Jika Degree vertex lebih besar dibanding vertex dengan degree sebelumnya akan di masukan indeks vertex yang baru
        *idMaks = (degree[*idMaks]< degree[i]) ? i : *idMaks;
    }
}

// Prosedur Print Degree
void PrintDegree(int N, int degree[]){
    // Print Degree Tiap Vertex
    for (int i = 0 ; i<N; i++) printf("DEGREE %d %d\n", i, degree[i]);
}

// Prosedure Print Max Vertex
void PrintMaxVertex(int idMaks){
    // Print indeks Max vertex
    printf("MAX_VERTEX %d\n", idMaks);
}

// Prosedur Print Isolated (Vertex tanpa degree)
void PrintIsolated(int N, int degree[]){
    // Jika terisolasi makas isolated = 1, jika tidak maka isolated = 0
    int isolated = 0;
    printf("ISOLATED");
    // Cari vertex yang terisolasi
    for (int i = 0 ; i<N; i++){
        if (degree[i]==0){
            printf(" %d", i);
            isolated = 1; // Ubah menjadi 1 jika vertex terisolasi
        }
    }
    // Jika tidak ada yang terisolasi, maka print NONE
    if (!isolated) printf(" NONE");
}



int main(void) {
    int N;
    scanf("%d", &N);
    int degree[N], idMaks = 0;

    ScanInput(N, degree, &idMaks);
    PrintDegree(N, degree);
    PrintMaxVertex(idMaks);
    PrintIsolated(N, degree);

    return 0;
}

```